

*Chen-Notation: $\square$ Entitätstyp $\cdot$ $\diamond$ Beziehungstyp $\cdot$ $\bigcirc$ Attribut
(unterstrichen=Schlüssel, gestrichelt=abgeleitet); Zahlen an Kanten = Kardinalität
aus Sicht des anderen Typs.*

→ Kette aus vier Entitätstypen, jeweils 1:n verbunden (siehe Diagramm).

→ Kette aus vier Entitätstypen, jeweils 1:n verbunden (siehe Diagramm).
 Region(ID,Name) —1:n „ist Teil von“— Land(ID,Name) —1:n „liegt in“—
 Standort(ID,Bundesland,Stadt,PLZ,Straße) —1:n „ist angesiedelt in“—
 Abteilung(ID,Name).



Abteilung über mehrere Standorte: 1:n → n:m, „ist angesiedelt in“; braucht eigene Relation `abteilung_standort`(`abteilung_id`, `standort_id`), optional mit Attribut (z.B. MA-Anzahl je Standort).

Beliebig tiefe Geo-Hierarchie: feste Kette unzureichend (nicht jedes Land hat Regierungsbezirke). Lösung: ein generischer Typ **Geografische Einheit**(ID,Name,Typ) mit **rekursiver 1:n** „ist Teil von“ auf sich selbst → Baum beliebiger Tiefe ohne Schemaänderung.

→ Mitarbeiter(ID,Vorname,Nachname,E-Mail,Telefon,Einstelldatum,Gehalt,Provisionssatz). „gehört zu“ Abteilung = 1:n.
Vorgesetzter = **rekursive 1:n** auf Mitarbeiter (Rolle Vorgesetzter=1, Rolle unterstellt=n).

Projekt(ID,Name): MA↔Projekt = **n:m** „arbeitet mit“; Attribute an der Beziehung: Anteil, Projektleitung.

- Job**(ID, Titel): Job $\leftrightarrow$ MA = 1:n „ist angestellt als“. Mindest-/Maximalgehalt = abgeleitete Attribute (gestrichelt), aus MA-Gehältern berechenbar.

→ Bezieht sich gleichzeitig auf MA+Abteilung+Job; keine paarweise Kombination ist eindeutig → eigener **schwacher Entitätstyp** (Doppelrahmen) mit drei 1:n-Beziehungen (war Kandidat für / wurde ausgeübt in / als). Attribute: `Start_Datum`, `End_Datum`. Dieses vollständige ER-Modell ist Basis für Aufgabe 2.



2.1 Entitätstypen → Relationen (Schlüssel = PK):

```
country(country_id, country_name)
location(location_id, street_address, postal_code, city, state_province)
department(department_id, department_name)
employee(employee_id, first_name, last_name, email, phone_number, hire_date, salary,
commission_pct)
job(job_id, job_title, min_salary, max_salary)
project(project_id, project_name)
```

min/max salary trotz Ableitbarkeit als materialisierte Spalten (SQL kennt keine autom. Attribute ohne Trigger/View).

```
country.region_id → region · location.country_id → country · department.location_id → location
```

- n:m:** employee2project(employee_id, project_id, assoc_pct, is_project_lead) — PK = beide FK.
- assoziativ:** job_history(employee_id, start_date, end_date, job_id, department_id) — PK laut Aufgabe = (employee_id, start_date).

region	country
region_id INT - region_name VARCHAR(25)	country_id CHAR(2) - country_name VARCHAR(40) - region_id INT→
location	department
location_id INT - street_address V(40) - postal_code V(10) - city V(30) - state_province V(25) - country_id CHAR(2)→	department_id INT - department_name V(30) - location_id INT→
job	project
job_id V(10) - job_title V(35) - min_salary DEC(8,0) - max_salary DEC(8,0)	project_id INT - project_name V(50)
employee	
employee_id INT - first_name V(20) - last_name V(25) - email V(25) - phone_number V(20) - hire_date DATE - job_id V(10)→ - salary DEC(8,2) - commission_pct DEC(2,2) - manager_id INT→ (self) - department_id INT→	
job_history	employee_projects
job_history_id INT - start_date DATE - end_date DATE - job_id V(10)→ - department_id INT→	employee_id INT - project_id INT→ - assoc_pct DEC(3,2) - project_lead BOOL

3.1.1 Warum nur zusammengesetzte Schlüssel die 2.NF verletzen?

→ 2.NF = volle funkt. Abh. jedes Nicht-Schlüsselattr. vom ganzen Kandidaten.
Partielle Abh. setzt echte, nichtleere Teilmenge des Schlüssels voraus. Einwertiger Schlüssel hat keine solche Teilmenge → 2.NF-Verstoß strukturell unmöglich.

RNr $\rightarrow$ KDNr, Pos, Datum, Betrag; KDNr $\rightarrow$ Name, Wohnort; transitiv RNr $\rightarrow$ Name, Wohnort.

(RNR → KDNr → Name, Wohnort) = Nicht-Schlüssel bestimmt Nicht-Schlüssel

- Änderungsanomalie:** Umzug → Wohnort in allen Rechnungszeilen ändern, sonst inkonsistent.

Löschanomalie: letzte Rechnung löschen → Stammdaten verloren.

3.4 BCNF. BCNF: linke Seite jeder nichttriv. FD ist Superschlüssel. In Kunde ist KDNr Schlüssel, in Rechnung ist RNr Schlüssel → beide bereits BCNF (Annahme: KDNr bestimmt nicht RNr). Zerlegung aus 3.3 genügt.

Prüfung) → autom. bis BCNF. Kapitel/Raumbelegung/schreibt: nur ein Nicht-

Schlüsselattr. bzw. voll abhängig → keine partielle/transitive Abh.; hört: nur Schlüsselattribute → trivial BCNF. **Ergebnis: gesamt BCNF.**

von zusammengesetztem Schlüssel abhängig → **BCNF**. Sonderfälle: (1) `postal_code-city` wäre transitiv, gilt aber international nicht global (fehlende PLZ) → kein Verstoß, nur Trade-off. (2) `job.min/max_salary` aus `employee.salary`

abgeleitet → relationenübergreifende Redundanz, kein BCNF-Verstoß innerhalb job.

Aufgabe 4 · DDL Teil 1

4.2 Schema (idempotent).

```
DROP DATABASE IF EXISTS hr;
CREATE DATABASE hr; USE hr;
```

DROP...IF EXISTS → Skript beliebig oft ausführbar.

4.3 Tabellen/Datentypen (Auszug).

```
CREATE TABLE region(
  region_id INT NOT NULL,
  region_name VARCHAR(25),
  PRIMARY KEY(region_id));
CREATE TABLE location(
  location_id INT NOT NULL AUTO INCREMENT,
  street_address VARCHAR(40), postal_code VARCHAR(12),
  city VARCHAR(30) NOT NULL, state_province VARCHAR(25),
  country_id CHAR(2) NOT NULL, PRIMARY KEY(location_id));
CREATE TABLE job_history(
  employee_id INT NOT NULL, start_date DATE NOT NULL,
  end_date DATE NOT NULL, job_id VARCHAR(10) NOT NULL,
  department_id INT NOT NULL,
  PRIMARY KEY(employee_id,start_date));
CREATE TABLE employee2project(
  employee_id INT NOT NULL, project_id INT NOT NULL,
  assoc_pct DECIMAL(3,2), is_project_lead BOOL,
  PRIMARY KEY(employee_id,project_id));
```

Pflichtattribute (Namen, Daten, E-Mail) NOT NULL; optionale (commission_pct, manager_id, des CEO, department_id) nullable. country/department/job/employee analog.

4.4.1 Fremdschlüssel + Referenzoptionen.

```
-- strukturelle Hierarchie: ON UPDATE CASCADE ON DELETE RESTRICT
ALTER TABLE country ADD FOREIGN KEY(region_id) REFERENCES
  region(region_id) ON UPDATE CASCADE ON DELETE RESTRICT;
ALTER TABLE location ADD FOREIGN KEY(country_id) REFERENCES
  country(country_id) _RESTRICT;
ALTER TABLE department ADD FOREIGN KEY(location_id) REFERENCES
  location(location_id) _RESTRICT;
ALTER TABLE employee
  ADD FOREIGN KEY(job_id) REFERENCES job(job_id) --
  RESTRICT;
  ADD FOREIGN KEY(department_id) REFERENCES department(department_id) --
  RESTRICT;
  ADD FOREIGN KEY(manager_id) REFERENCES employee(employee_id) --
  RESTRICT;
-- Detail-/Historientabellen: ON DELETE CASCADE
ALTER TABLE job_history ADD FK(employee_id)=employee_fk(job_id)=job,
  FK(department_id)=department ON UPDATE CASCADE ON DELETE CASCADE;
ALTER TABLE employee2project ADD FK(employee_id)=employee,
  FK(project_id)=project ON UPDATE CASCADE ON DELETE CASCADE;
```

4.4.2 Verständnis.

- **RESTRICT employee.department_id:** Abteilung mit MA nicht löschar (FK-Fehler) → erst MA umziehen.
- **UPDATE location_id:** ON UPDATE CASCADE → neuer Wert automatisch in department übernommen.
- **DELETE location mit Departments:** abgelehnt (RESTRICT).
- **DELETE Mitarbeiter:** ist Manager (von manager_id referenziert) → RESTRICT blockt; sonst erlaubt; job_history & employee2project werden per CASCADE mitgelöscht.
- **Alle CASCADE, DELETE Region:** Kettenreaktion
Region→Countries→Locations→Departments→Employees(rekursiv)→job_history/e2p → löscht Großteil der DB. Deshalb bewusst RESTRICT für Struktur-FKS & manager_id, CASCADE nur für Historientabellen.

Aufgabe 5 · Constraints

5.1 Arithmetik / 5.2 Reguläre Ausdrücke.

```
ALTER TABLE job ADD CONSTRAINT MinSalaryPositive CHECK(min_salary>0),
  ADD CONSTRAINT MaxSalaryGreaterThenMinSalary
  CHECK(max_salary=>min_salary);
ALTER TABLE employee
  ADD CONSTRAINT SalaryPositive CHECK(salary>0),
  ADD CONSTRAINT commissionPctPositive CHECK(commission_pct>0),
  ADD CONSTRAINT EmailPattern CHECK(email RLIKE '^([a-zA-Z][a-zA-Z0-9]*@[a-zA-Z]+\.[a-z]{2,4})$');
ALTER TABLE job_history ADD CONSTRAINT EndDateAfterStartDate
  CHECK(end_date>start_date);
ALTER TABLE employee2project ADD CONSTRAINT AssocPctRange
  CHECK(assoc_pct>0 AND assoc_pct<=1);
ALTER TABLE location ADD CONSTRAINT AtLeastOneDigitInStreetAddress
  CHECK(street_address RLIKE '[0-9]');
```

E-Mail-Regex: ^...\$ verankern; erstes Zeichen Buchstabe; dann alphanum.; @; Buchstaben+...; 2–4-stellige TLD klein.*

5.3 Komplexe Constraints. Einfacher CHECK sieht nur die eigene Zeile → zeilen-/tabellenübergreifend braucht TRIGGER.

- MA ≠ eigener Vorgesetzter: noch als CHECK CHECK(employee_id<>manager_id) .
- Gehalt im Job-Rahmen (Blick in job) → BEFORE INSERT-Trigger, SELECT min/max_salary, sonst SIGNAL SQLSTATE '45000'
- 2 assoc_pct je MA ≤ 1 → Trigger mit SUM über employee2project (bewusst nicht erzwungen, s. 8.4.1).
- job_history: Zeiträume je MA überlappungsfrei & start_date ≥ hire_date → ebenfalls Trigger.

```
DELIMITER $$
CREATE TRIGGER trg_salary_in_range BEFORE INSERT ON employee FOR EACH ROW
BEGIN DECLARE v_min,v_max DECIMAL(8,0);
  SELECT min_salary,max_salary INTO v_min,v_max FROM job WHERE
  job_id=NEW.job_id;
  IF NEW.salary>max THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT='Gehalt ausserhalb Jobrahmen';
  END IF;
END$$ DELIMITER ;
```

Aufgabe 6 · DML

6.1 Einfügen.

```
INSERT INTO region VALUES (1,'Europe'),(2,'Americas'),(3,'Asia'),
  (4,'Middle East and Africa');
INSERT INTO country VALUES ('IT','Italy'),('JP','Japan'),(3),
  ('US','United States of America'),('CA','Canada'),(2),('EG','Egypt'),(4);
-- Projektzuordnungen (employee_id,project_id,anteil,leitung)
INSERT INTO employee2project VALUES (186,102,0.70,FALSE); --
  Dellinger-Lean Mgmt
INSERT INTO employee2project VALUES (157,100,1.00,TRUE); -- Sully-US
  Sales (Leitung)
INSERT INTO employee2project VALUES (120,101,0.20,FALSE); -- Weiss-Europe
  Sales
```

6.2 Aktualisieren.

```
UPDATE employee SET phone_number='650.127.3418'
  WHERE first_name='Michael' AND last_name='Rogers';
UPDATE job SET min_salary=4000 WHERE min_salary<4000;
UPDATE employee SET salary=salary*1.03;
```

6.3 Löschen.

```
DELETE FROM employee2project WHERE assoc_pct<=0.25;
DELETE FROM employee WHERE first_name='Jonathon' AND last_name='Taylor';
```

job_history(employee_id) ON DELETE CASCADE → Taylors 2 History-Zeilen (SA_REP, SA_MAN) verschwinden automatisch (vorher 2, nachher 0 Zeilen).

6.4 Constraints testen. je Constraint 1x gültig & 1x verletzend, z.B.:

```
INSERT INTO job VALUES ('OK','x',3000,6000); -- gültig
INSERT INTO job VALUES ('F1','x',-100,6000); -- MinSalaryPositive
  verletzt;
INSERT INTO job VALUES ('F2','x',5000,3000); -- Max>=Min verletzt
-- salary<0, end_date
```

Aufgabe 7 · SELECT Grundlagen

- 7.1 SELECT country_name FROM hr.country;

- 7.2 Uhrzeit HH:MM SELECT DATE_FORMAT(NOW(),'%H:%i') FROM DUAL;

- 7.3 DISTINCT SELECT DISTINCT manager_id FROM hr.employee WHERE manager_id IS NOT NULL;

- 7.5 CEO SELECT * FROM hr.employee WHERE manager_id IS NULL;

- 7.6 _WHERE min_salary>2000 AND LOWER(job_id) LIKE '%clerk%';

- 7.7 Manager (Subquerv) _WHERE employee_id IN (SELECT DISTINCT manager_id FROM hr.employee WHERE manager_id IS NOT NULL);

7.4 String-Format Adresse.

```
SELECT street_address AS Adresse,
  IFNULL(CONCAT(country_id,' - ',postal_code,' ',city,'
  ('',state_province,'))','') AS Telefon,
  ('unbekannt') AS Stadt
FROM hr.location;
```

7.8 Komplex (Manager, Gehalt>10000, vor 1996 eingestellt).

```
SELECT CONCAT(first_name,' ',last_name) AS Name,
  REPLACE(LOWER(email),'@example.com','@...') AS EMail,
  SUBSTR(phone_number,3) AS Telefon,
  CONCAT(DATE_FORMAT(hire_date,'%d.%m.%Y'),' ('',
  ROUND(DATEDIFF(STR_TO_DATE('01.01.2023','%d.%m.%Y'),hire_date)/365,1),'
  Jahre')) AS Einstellungsdatum,
  CONCAT(ROUND(salary/1000,0),'k EUR') AS Gehalt,
  IFNULL(manager_id,'nemand') AS Vorgesetzter
FROM hr.employee
WHERE salary>10000 AND hire_date
```

Aufgabe 8 · SELECT Teil 2

- 8.1.1 Telefonbuch SELECT last_name,first_name,phone_number FROM hr.employee ORDER BY last_name,first_name ASC;

- 8.1.2 Rückwärts SELECT phone_number,last_name,first_name _ ORDER BY phone_number ASC;

- 8.2.2 Standorte/Land SELECT country_id,COUNT(location_id) c FROM hr.location GROUP BY country_id ORDER BY c DESC;

8.3.1 Abteilungsdetails (4er-Join).

```
SELECT region.region_name, country.country_name,
  location.state_province,
  CONCAT(location.postal_code,' ',location.city,'
  ',location.street_address) AS Adresse,
  department.department_name
FROM hr.department
JOIN hr.location ON department.location_id=location.location_id
JOIN hr.country ON location.country_id=country.country_id
JOIN hr.region ON country.region_id=region.region_id;
```

8.3.2 MA=Projekt.

```
SELECT project.project_name, employee.first_name, employee.last_name,
  employee2project.assoc_pct
FROM hr.project
JOIN hr.employee2project ON
  project.project_id=employee2project.project_id
JOIN hr.employee ON
  employee.employee_id=employee2project.employee_id;
```

8.4.1 Überlastete MA (>100%).

```
SELECT employee.last_name Nachname, employee.first_name Vorname,
  SUM(employee2project.assoc_pct) Gesamtauslastung,
  GROUP_CONCAT(project.project_name SEPARATOR ', ') Projekte
FROM hr.project
JOIN hr.employee2project ON
  project.project_id=employee2project.project_id
JOIN hr.employee ON employee.employee_id=employee2project.employee_id
GROUP BY employee.first_name,employee.last_name HAVING
  Gesamtauslastung>1;
```

Liefert Baer(Hermann)[1,10]Europe+US Sales Campaign (E-Constraint aus 5.3 bewusst nicht als CHECK).

Aufgabe 9 · Mengenoperationen & Views

9.1 UNION (3 ärmste ≤1996 + 3 ärmste gesamt).

```
(SELECT employee_id,first_name,last_name,hire_date,job_id,salary FROM
  hr.employee
  WHERE DATE_FORMAT(hire_date,'%Y')<=1996 ORDER BY salary ASC LIMIT 3)
UNION
(SELECT employee_id,first_name,last_name,hire_date,job_id,salary FROM
  hr.employee
  ORDER BY salary ASC LIMIT 3)
ORDER BY salary ASC;
```

UNION entfernt Duplikate → höchstens 6 (ggf. weniger) Zeilen.

9.2 View employee_details.

```
CREATE OR REPLACE VIEW employee_details AS
SELECT
  employee_id,job_id,manager_id,department_id,location_id,country_id,
  first_name,last_name,salary,commission_pct,
  department_name,job_title,city,state_province,country_name,region_name
FROM hr.employee
JOIN hr.department USING(department_id)
JOIN hr.job USING(job_id)
JOIN hr.location USING(location_id)
JOIN hr.country USING(country_id)
JOIN hr.region USING(region_id);
SELECT * FROM employee_details;
```

USING(...) möglich, da FK-Spalten gleich benannt → gemeinsame Spalte erscheint nicht doppelt.